Session 7

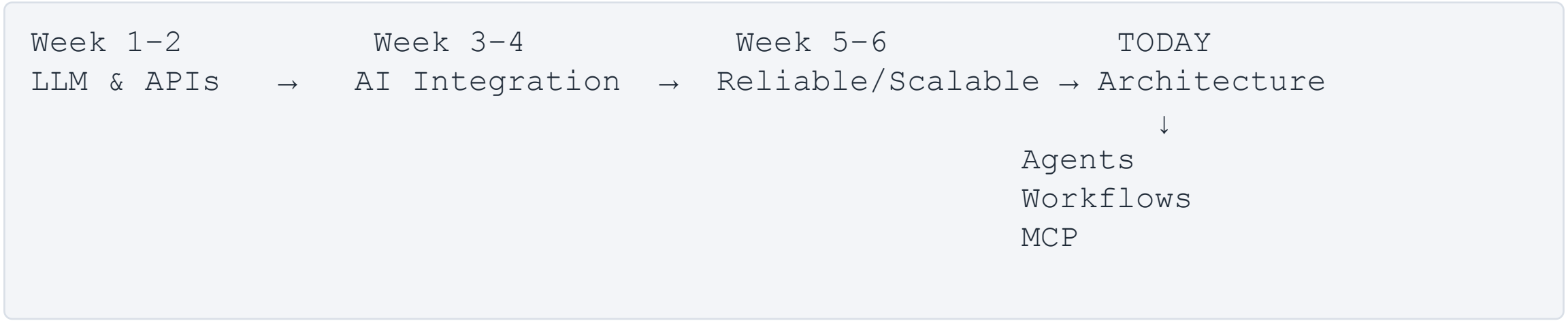
Agentic Software Architecture

Multi-Agent Systems, Workflows & Model Context Protocol (MCP)

Learning outcomes

- Explain agents, tools, workflows, and MCP.
- Compare single-agent and multi-agent architectures.
- Understand orchestration and routing.
- Design MCP-based tool integrations.
- Apply agentic patterns to the capstone architecture.

Where We Are in the Course



You've built an AI feature. Today we ask: how should the whole AI-enabled system be structured?

The AI Applications We Have Built So Far

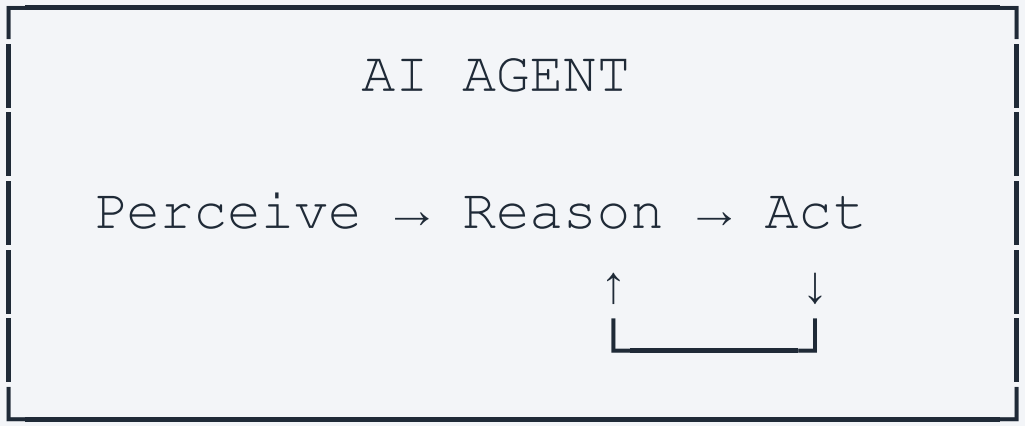
```
User → Application → LLM API → Response
```

What happens when the application needs to retrieve data, calculate something, send an email, query a database, or perform an action?

```
                        ┌→ Database
                        ├→ External API
LLM Application ────────├→ Files
                        ├→ Search
                        └→ Business Logic
```

This creates the motivation for tools and agents.

What Is an AI Agent?



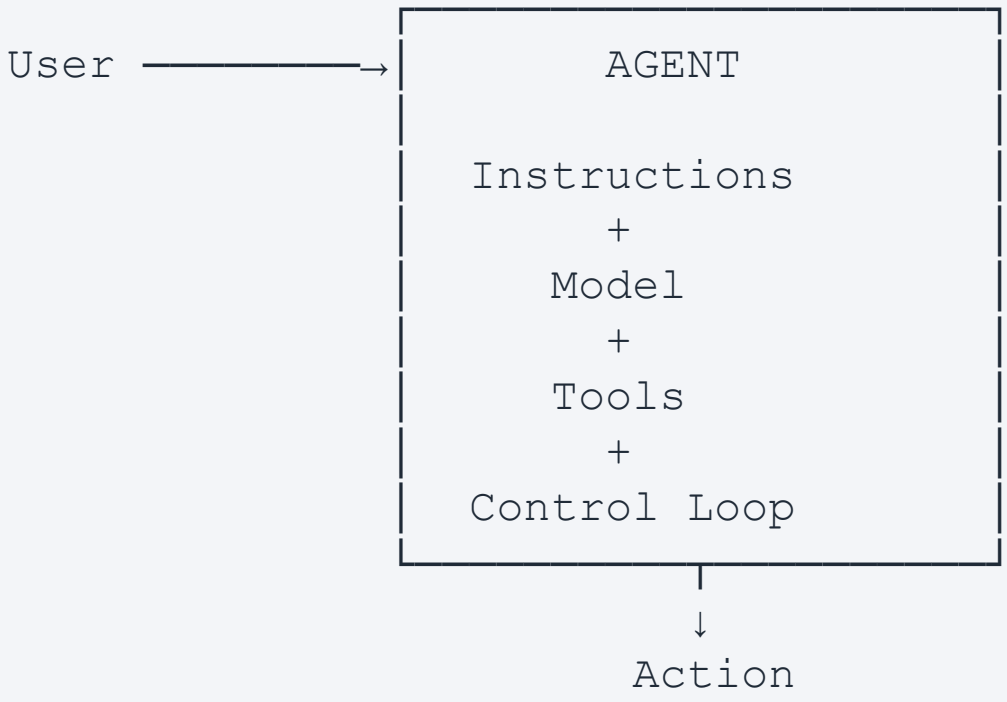
Agents can perceive inputs, reason about actions, and act through tools and APIs.

LLM vs AI Agent

LLM	AI Agent
Generates text	Pursues a task / goal
Responds to a prompt	Decides what action to take
Limited to its context	Can access external systems
Cannot inherently perform actions	Invokes tools
Usually one input/output cycle	May execute multiple steps

LLM = reasoning component. Agent = software system around the LLM.

Anatomy of an Agent



What Is a Tool?

```
def get_course(course_code: str):
    ...
```

A good tool has:

```
Name        → get_course
Description → Retrieve course information
Input       → course_code: string
Output      → course information
```

LLM decides what should happen. Tool performs the operation.

Why Should the Tool Do the Work?

Bad

```
"What is the GPA of A, B+, B, C?"
LLM → calculates → maybe correct
```

Better

```
User → Agent → calculate_gpa() → Deterministic Code → 3.18 → Agent
explains
```

Use AI where reasoning is useful. Use deterministic software where correctness is required.

Tool Calling: Step by Step

```
USER          "Find information about ICT304."
   ↓
LLM           "I need course information."
   ↓
Select tool   get_course
   ↓
Arguments     {"course_code": "ICT304"}
   ↓
TOOL EXECUTES
   ↓
Tool result   {"name": "Mobile Development II", "credits": 3}
   ↓
LLM  →  Final Response
```

Distinguish reasoning (LLM) from execution (tool).

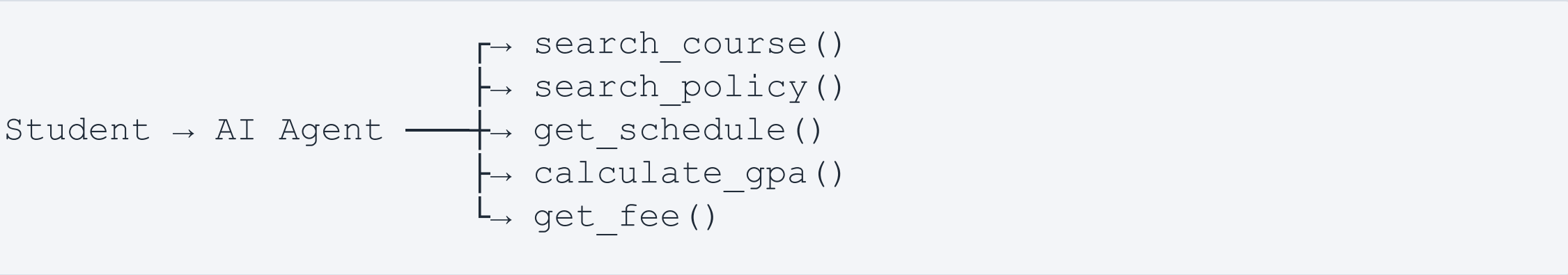
Quick Check: LLM or Tool?

Which should be performed by the LLM, and which by a tool?

1. Decide whether a question concerns finance or academics.
2. Retrieve a student's current balance.
3. Explain why the balance is outstanding.
4. Update the student's payment status.

```
1 → LLM / routing          3 → LLM
2 → Tool                   4 → Tool
```

Start With One Agent



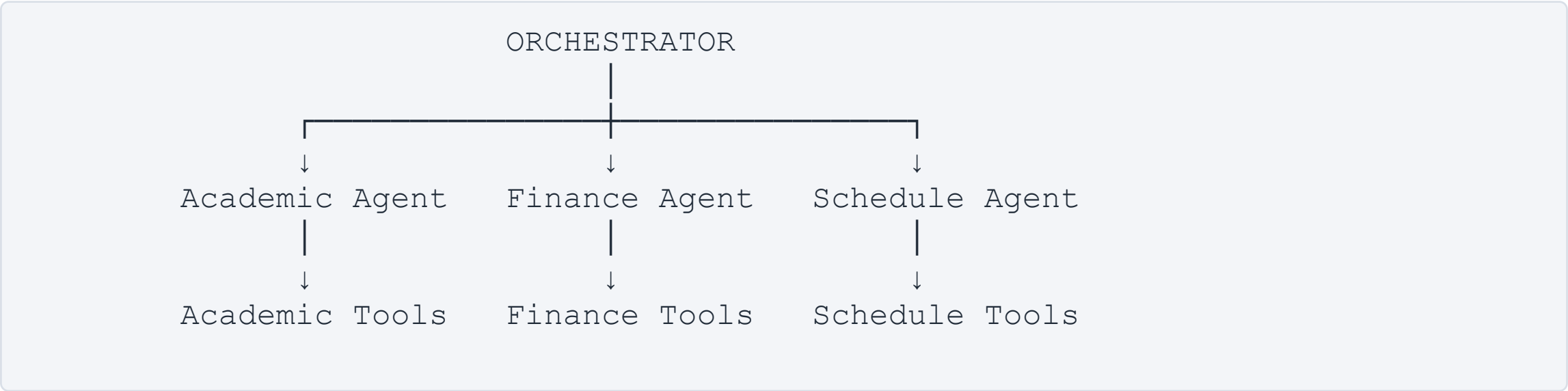
This is often enough. Don't introduce multiple agents merely because they are available.

When One Agent Becomes Too Complex

```
ONE AGENT
  20 tools · 10 responsibilities
  Large instructions
  Different permissions
  Different data sources
  Different failure modes
        ↓
  Harder to reason about · harder to test · harder to maintain
```

Then introduce specialization.

Multi-Agent System



Different agents can have different instructions, models, tools, permissions, and responsibilities.

The Orchestrator

Who decides which agent gets the request?

```
"How much is my tuition?"   →   Orchestrator   →   Finance Agent
"When is ICT304?"           →   Orchestrator   →   Schedule Agent
```

Routing is an architectural responsibility.

Three Routing Strategies

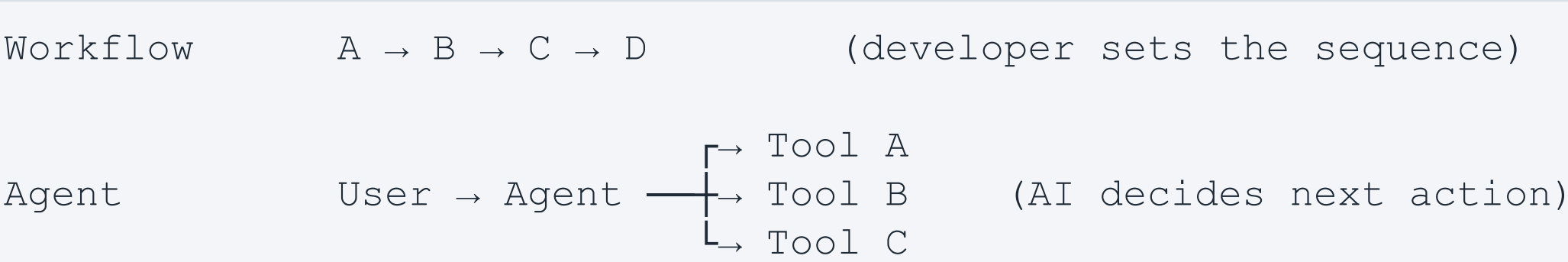
```
Rule-based     if category == "finance":
                   finance_agent()

LLM-based      Query  →  Router LLM  →  Agent

Hybrid         Known / critical request → Rules
               Ambiguous request        → LLM
```

Which would you prefer for a university payment system? Why?

Workflow ≠ Agent



When Should We Use Each?

Situation	Pattern
Steps are known	Workflow
Dynamic decision needed	Agent
Many related tools	Single agent
Clearly separated responsibilities	Multi-agent
Reliability + dynamic reasoning	Hybrid

```
Reset password                → workflow
Investigate unusual activity → agent
Generate monthly invoice      → workflow
Research why sales declined   → agent
```

Architecture Is About Control

Where should your application sit on this spectrum?

More autonomy is not automatically better architecture.

The Integration Problem

```
AI Agent
  ├── Google Drive API        ├── File System
  ├── GitHub API              ├── University API
  ├── PostgreSQL              └── Weather API
```

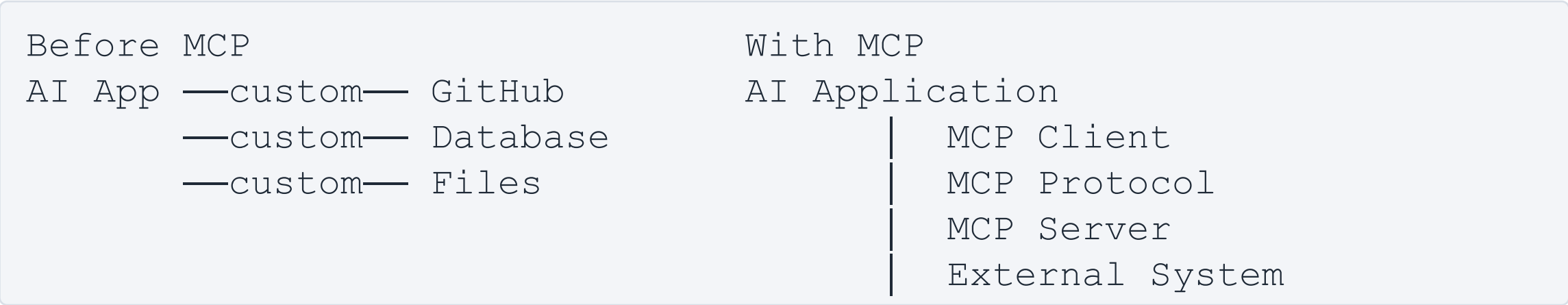
Every integration requires custom code:

```
Agent ↔ GitHub adapter        Agent ↔ Files adapter
Agent ↔ DB adapter            Agent ↔ University adapter
```

What happens when we have 10 agents × 20 integrations?

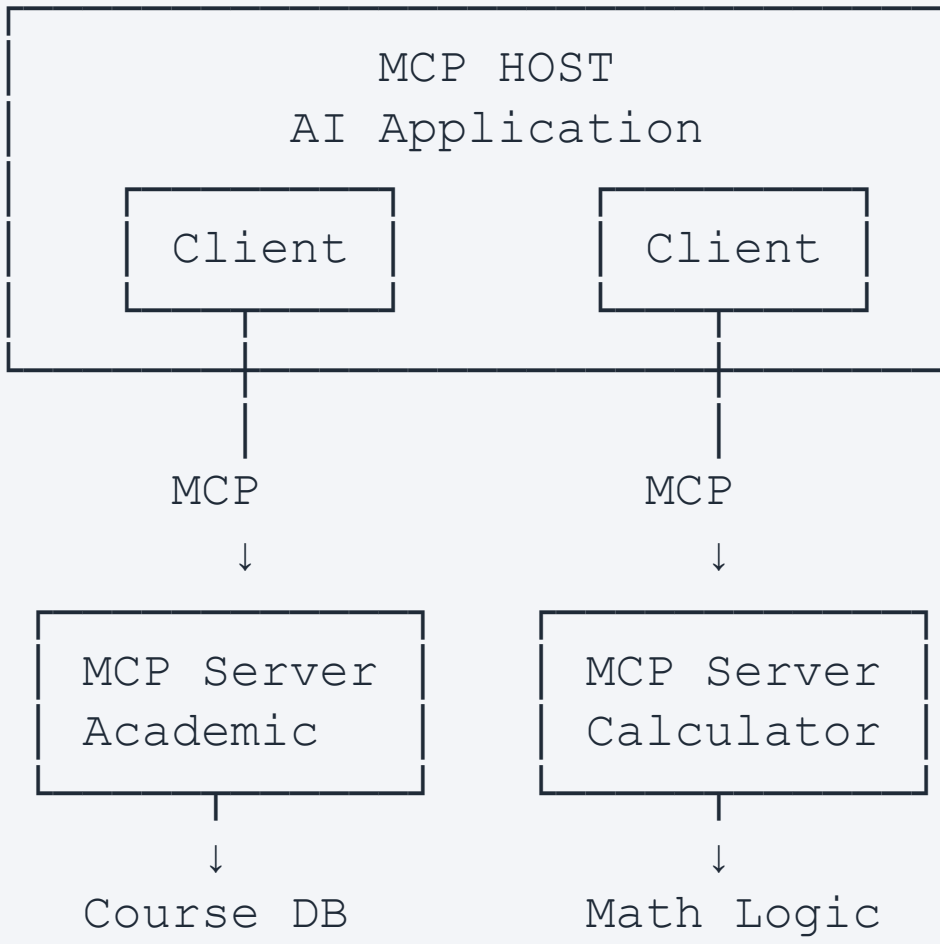
Model Context Protocol (MCP)

MCP provides a standardized way for AI applications to connect to external capabilities and context.



Standardized interfaces · discoverability · modularity · separation of concerns.

MCP Architecture



MCP Server Capabilities

```
MCP SERVER
    ├── Tools        → Actions
    ├── Resources    → Context / Data
    └── Prompts      → Reusable templates
```

Our lab will primarily focus on Tools.

Tool Discovery: Why MCP Is Different

Hard-coded integration

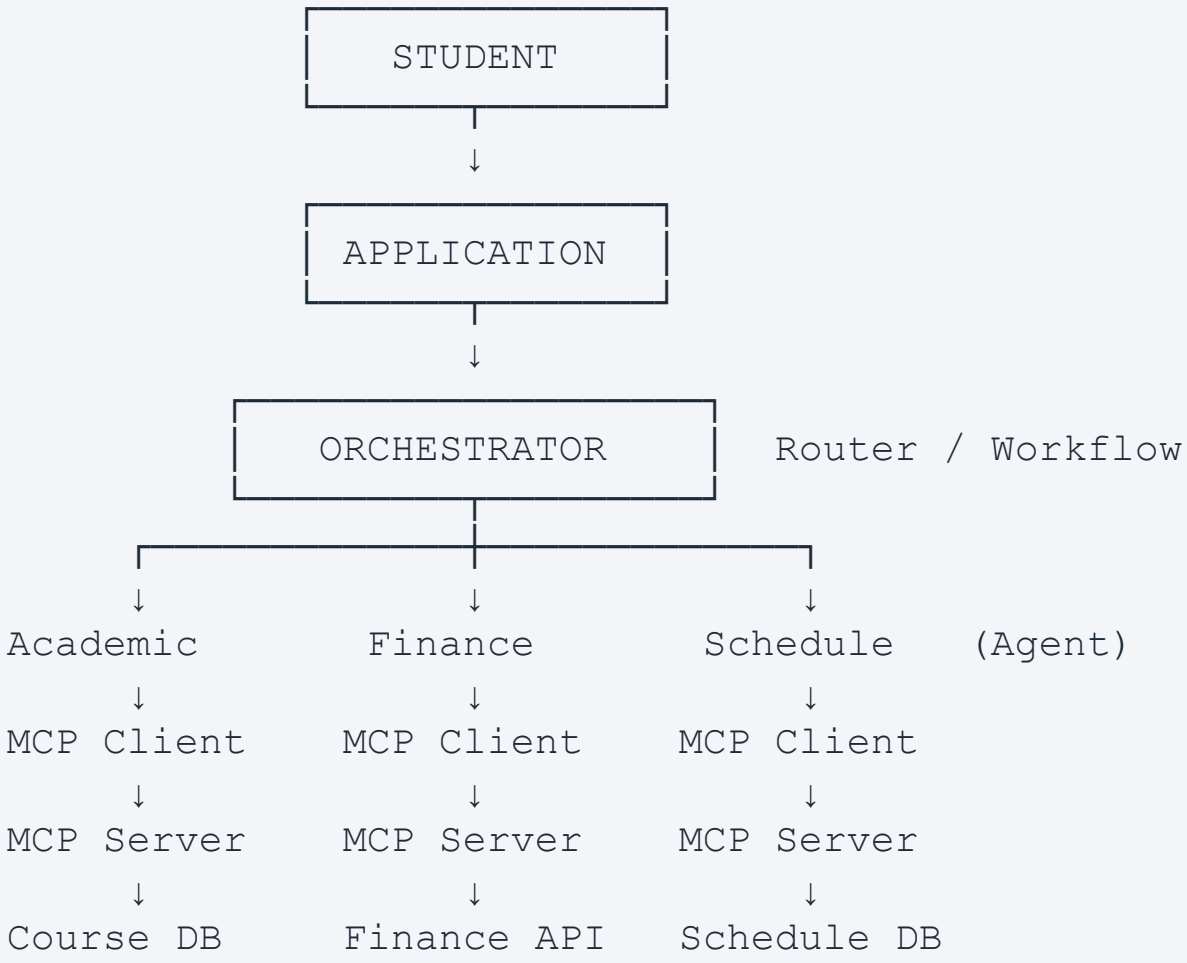
```
if operation == "course":
    call("/course")
```

MCP-style interaction

```
Agent → MCP Client → "What tools are available?" → tools/list
      → MCP Server → tool definitions + schemas
```

LLM selects tool → tools/call → Server executes → structured result

Complete Agentic Architecture



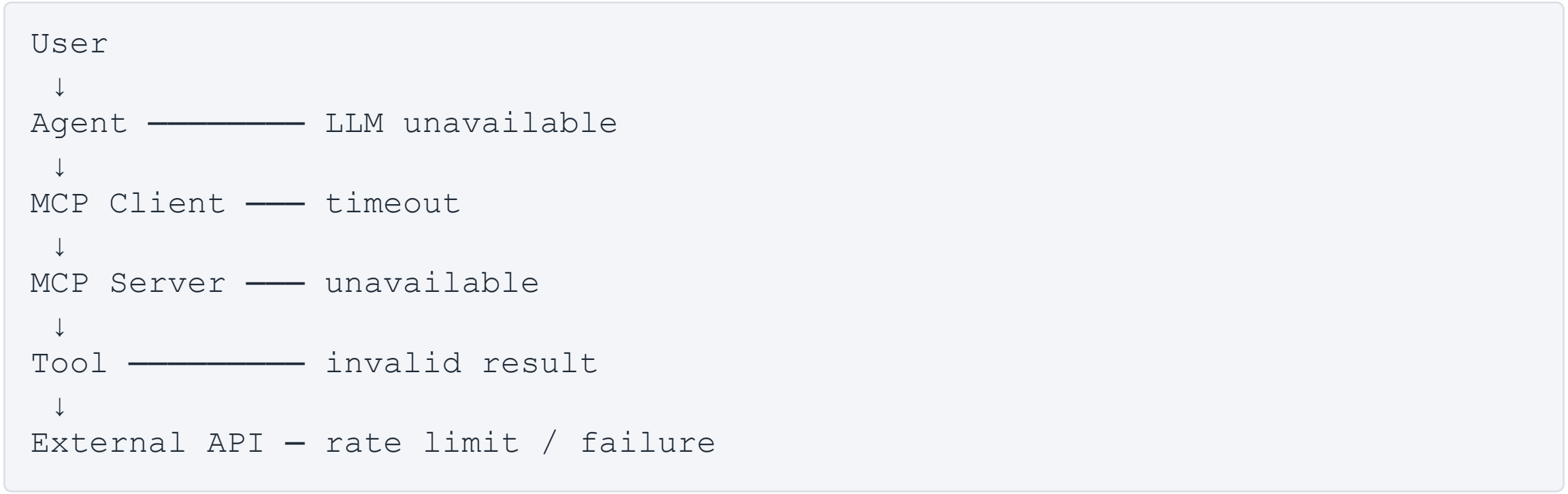
Identify: reasoning · routing · deterministic execution · protocol boundary · failure points.

Trace One Request

prompt: "What is ICT304, and do I have the prerequisite to take it?"

```
1. User sends question
2. Orchestrator → Academic Agent
3. Agent discovers / selects tools
4. get_course("ICT304")
5. check_prerequisite("ICT304")
6. Tools return structured data
7. Agent reasons over results
8. Agent generates explanation
9. User receives answer
```

What Can Go Wrong?



Where should we handle each failure? (Connect back to Weeks 5 and 6.)

Design for Failure

```
Timeout · Retry · Fallback · Validation
Logging · Human approval · Graceful degradation
```

Example — Academic MCP unavailable

```
BAD:    500 Internal Server Error

BETTER: "I can't access the course catalogue right now.
         Please try again shortly."
```

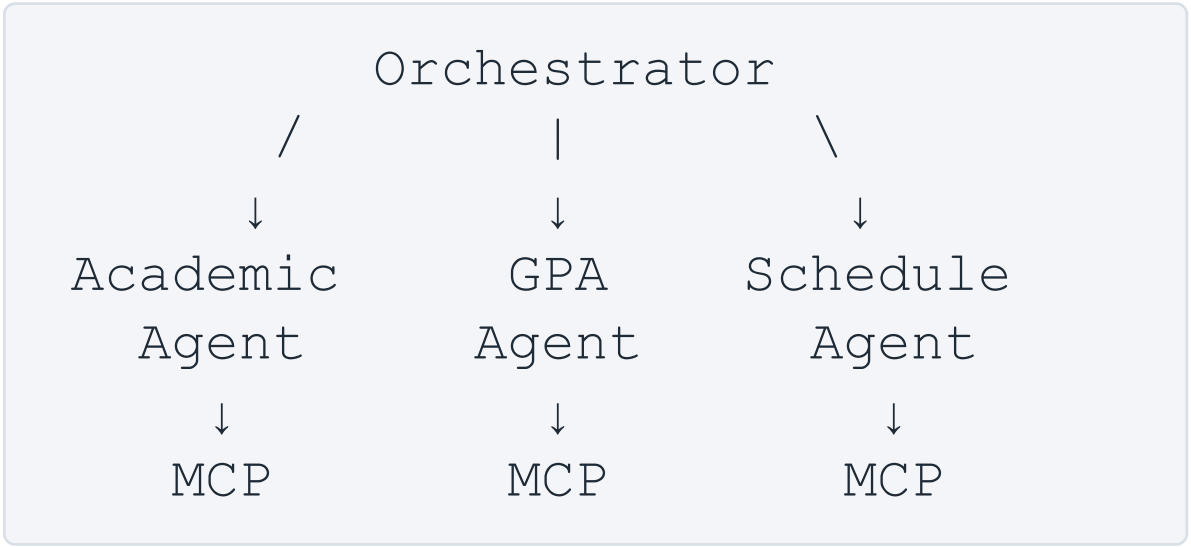
Architecture Challenge

Build an AI University Assistant

- Answer course questions.
- Search policies.
- Calculate GPA.
- Check schedules.
- Explain answers conversationally.

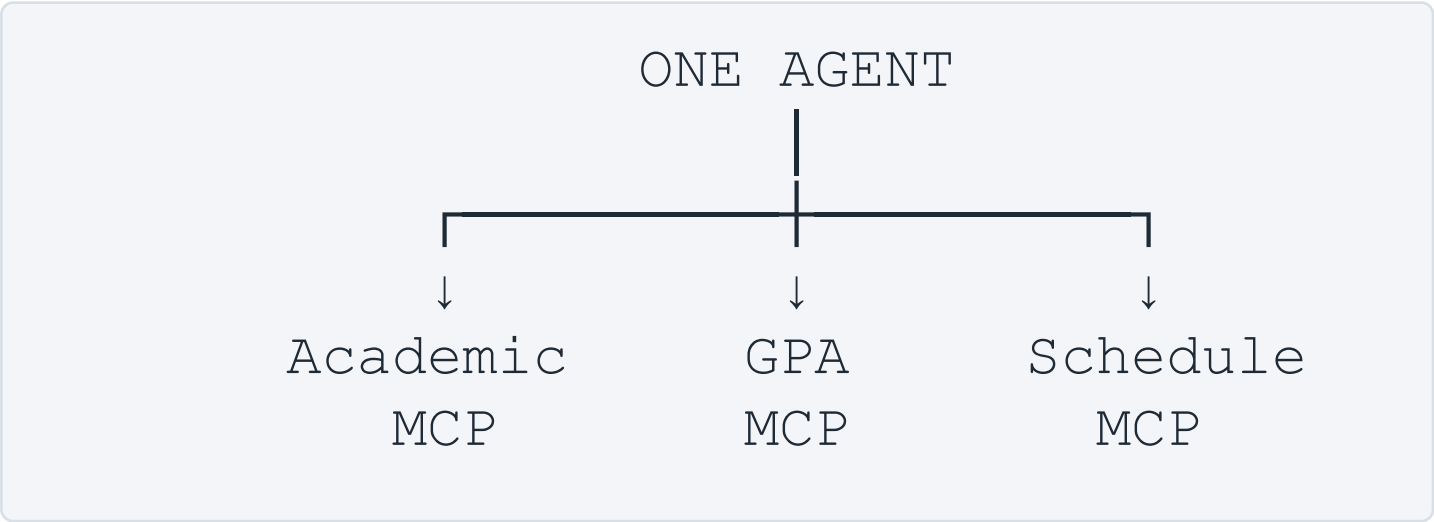
```
How many agents?     What tools?     What MCP servers?
What stays deterministic?     Where is the orchestrator?
```

Architecture A: Multi-Agent



Is this good architecture?

Architecture B: Simpler



Do we actually need three agents?

When Should You Use Multi-Agent?

Use multiple agents when there is a genuine reason:

- Different responsibilities
- Different tool permissions
- Different domain expertise
- Different models
- Independent scaling
- Separate security boundaries

Don't use multiple agents just because "multi-agent sounds more advanced."

Design Principle

Start with the simplest architecture that satisfies the requirements.

```
Can deterministic code solve it?   → No ↓
Can a workflow solve it?           → No ↓
Can one agent solve it?            → No ↓
Consider multiple agents.
```

Today's Lab: Build a Student Assistant with MCP

```
User → Agent → MCP Client
                  ├── Academic MCP → Courses · Policies · Schedules
                  └── Math MCP     → GPA · Average
```

Students implement:

```
search_course()     search_policy()     get_schedule()
calculate_gpa()     calculate_average()
```

Then add routing / tool selection.

Lab Challenges

- Challenge 1 — Add check_prerequisite().
- Challenge 2 — Handle an unavailable MCP server.
- Challenge 3 — Support: "Find the credits for ICT304 and FESE307 and tell me the total."

Capstone Milestone: Architecture Design

```
Users → Application → AI / Orchestration → Agents / Workflows
      → Tools / MCP → Data / External Systems
```

Each team identifies:

- Agent — where reasoning occurs
- Workflow — where flow is deterministic
- Tools — what capabilities AI can invoke
- MCP — integration boundaries
- Data — authoritative information sources
- Failure handling — timeout / fallback / error boundaries

Why is this architecture appropriate for your capstone?

Final Takeaways

```
LLM → LLM + Tools → Agent → Agent + Multiple Tools
    → Workflow + Agent → Multi-Agent System → MCP-based Integration
```

1. Agents reason; tools execute.
2. Workflows provide control; agents provide flexibility.
3. More agents ≠ better architecture.